Team decision-tool project

OM 522, Fall 2026

Revised October 7, 2026

The project on one page

A team of one to three students builds one application that makes a recurring operating decision from files of operating data. The application solves the decision two ways (the option's published rule to beat and a researched method), checks both decisions independently, compares them, tests how the two plans hold up when the data turn out to be wrong, and recommends an action. Teams are expected to build with agentic coding tools, and every member must be able to explain what those tools produced. The project is worth 70 course points, awarded at a deterministic prototype (20 team points), a final submission (40 team points), and individual contribution measured by confidential peer evaluation (10 points).
Section 2 defines what every team builds, Section 3 lists the deliverables, and Section 4 publishes the rubric and the run the instructor performs on each final submission. The option brief adds what is specific to each option. This guide contains no deadlines, and the course schedule and Blackboard control all dates.

What changed in this revision

This revision removes work rather than adding it. A team that has already built something now optional may keep it. The two changes that can touch existing work are a loader that still requires a removed data file or column and, for a team whose baseline is not the rule to beat, implementing the rule.

- Replanning is no longer required. The final experiment is a stress test of both methods' fixed plans under 30 scenarios (Section 2.7).
- The rule to beat is now the first method. A team that already built a different baseline may keep it as an extra method but also implements the rule, which the worked example tests. The researched method must beat the rule on the held-out case.
- Input validation, tests, and the instructor's checking rules are shorter (Sections 2.2, 2.5, and 4.3). The "infeasible with evidence" status, the five-runs rule, and the hardware and determinism rules are gone.
- The researched method needs one published source rather than three, and a visual interface is optional.
- Each video now includes a walk-through of agent-drafted code by every member, and the rubric moves points toward the videos.
- Each brief names one secondary measure, and the data drop unused files and columns (option 2 `disruptions.csv`, option 7 `day_config.csv`, the empty locked columns in options 1 and 4, and `distance` and `fixed_cost` in options 3 and 7). Every other value is unchanged. Download the revised data ZIP.

1 Project overview

Each team builds a decision tool, i.e., a small application that a manager could run every time a recurring operating decision comes due. The deterministic prototype applies the first half of the course (dispatching rules, construction and improvement heuristics, and feasibility checks) to fixed data, and the final submission applies the second half (variability) to test how the prototype's plans hold up. Table 1 summarizes the seven options. Every option supplies synthetic data, a data dictionary, and a worked example, every option carries the same requirements and rubric, and more than one team may choose the same option.

Table 1: The seven project options.

#	Operating decision	What makes it hard	Most relevant meetings
1	Assign jobs to machines that can perform them, and order each machine's queue.	Each job can use only some machines, at machine-specific speeds.	LPT construction; local improvement; eligibility-constrained resources.
2	Schedule jobs whose operations follow different machine routes.	Operation order within each job, competing machines, and machine alternatives.	Critical Path Method; job-shop representation and feasibility.
3	Plan delivery routes for a vehicle fleet.	Vehicle capacity, customer time windows, and shift length.	TSP construction and improvement; Clarke–Wright Savings.
4	Schedule project activities that share crews and equipment.	Precedence plus resource limits that the critical path ignores.	Critical Path Method and precedence logic.
5	Group warehouse orders into picking trips and route each trip.	Batching saves walking but can make urgent orders late.	TSP construction and improvement; neighborhood search.
6	Choose one job sequence for an eight-stage production line.	Releases, weighted lateness, and waiting that builds up at later stages.	Dispatching rules; neighborhood search for weighted tardiness.
7	Choose which inspections a team completes and route each inspector.	Optional visits, limited shifts, time windows, and several inspectors.	TSP construction and improvement; Clarke–Wright Savings.

2 Common requirements

This section describes what every option requires, in the order a team typically builds it. Each option brief adds the option-specific constraints, measures, rule to beat, and uncertain inputs, and Section 4 shows how each requirement is scored.

2.1 The decision

The repository README opens with one paragraph, in the team's own words, that describes the decision the tool makes, who makes it, and any assumption the team added beyond the option brief. The brief already defines the constraints and measures, so the README does not restate them.

2.2 Input and output

The application reads any instance directory in the published format without code edits. Before solving, it checks that every required file and column is present, that values have their declared types, that every referenced identifier exists (e.g., every operation's job), and that processing times and durations are positive. Input that fails a check produces a message naming the file and column, and missing or invalid values are never silently replaced. The application leaves input files unchanged and writes its results to a separate output directory, which holds a decision file named `decision.json` and a short summary of the measures and the run status. The decision file uses the structure of the option's `worked-example/solution.json`.

2.3 Two methods

The first method is the option's rule to beat, implemented exactly as its brief states. The worked example in each data ZIP applies that rule to the hand-checkable case, so reproducing the example is the first method's test, and every member should be able to trace the rule by hand on that case. A team that already built a different baseline may keep it as an extra method.

The second method is a researched method, i.e., a method the team finds in at least one published source (the reading named in the brief qualifies) and explains why it fits the decision. Two runs of one method with different seeds or settings do not count as two methods. Either method may call a general optimization solver (e.g., HiGHS or OR-Tools CP-SAT) on a model the team formulates, but a package that already solves the option's problem (e.g., the OR-Tools routing solver or PyVRP) cannot serve as either method, although it may appear in the comparison as a labeled benchmark. Code that a team drafts with agentic coding tools counts as written for the project once the team has reviewed it and can explain it. Code from the in-class notebooks may be reused only for loading or plotting, with attribution in the README.

For each method, the README explains how a solution is represented, its construction or improvement steps, its stopping rule, any randomization and its seed, and its known limitations. A team that uses a solver also reports its completion status and time limit, because a solver that stops at its time limit has not established an optimal solution.

2.4 The rule to beat

Every option brief defines a rule to beat, i.e., a simple, fully specified construction rule with exact tie-breaking. At the final submission, the instructor runs the team's researched method on a held-out case that teams never see. The method beats the rule when its primary measure is better than the rule's by more than 0.001. The rule constructs a feasible decision on the hand-checkable case, every development case, and the held-out case, and the instructor has confirmed that the held-out case admits a decision that beats it. A researched method that cannot outperform a one-page rule on new data does not support a recommendation to adopt it.

2.5 Independent checking and tests

The solution checker is separate from both methods, i.e., it reads the input files and an exported decision file and recomputes everything from those two sources. It reports which jobs, operations, routes, batches, activities, or visits violate a constraint, and it recomputes the measures rather than trusting totals reported by a method. A feasible decision satisfies every required constraint, and a better measure never compensates for an infeasible decision. The checker is the team's main protection against agent-written code that claims a result it did not achieve.

Every submission includes three automated tests, which are (1) the rule to beat reproduces the worked example's decision and measures, (2) the supplied edge case produces the response that Section 4.3 gives for it ("no feasible decision found" in options 1 to 5, a rejection in option 6, and an empty plan in option 7), and (3) a copied instance with a required column deleted is rejected with a clear message. Running the tool on copied instances with reordered rows or renamed identifiers is a useful additional check.

2.6 Comparison on fixed data

Both methods receive the same input files and measure definitions. The deterministic comparison covers the hand-checkable case and the three development sizes and reports feasibility, the primary measure, the brief's secondary measure, and computation time for each method and instance. The prototype's initial comparison may use any three of these four instances, and the final submission covers all four. A comparison states the time limit and the computer used.

2.7 Stress test of fixed plans

The deterministic comparison assumes the data are exact. The final submission tests how each method's plan holds up when they are not. The team chooses one uncertain input from its option brief and states how it varies (e.g., each processing time multiplied by an independent lognormal factor with mean 1 and coefficient of variation 0.2). It generates 30 scenarios from a documented seed and saves them to files before running anything, so that both plans face identical realizations. Each method's deterministic plan on the medium development instance is then executed without changing any decision under every scenario, using the execution rule in the brief. For each method, the team reports the mean and 90th percentile, across the 30 scenarios, of the primary measure and of the brief's secondary measure, the number of scenarios in which the plan breaks a hard constraint (e.g., a missed time window), and a plot of per-scenario results.

2.8 The recommendation

The final recommendation names the recommended method, its expected benefit in the project's units, the tradeoff accepted to obtain it, and one condition under which the recommendation would change (e.g., processing-time variability above the levels tested).

2.9 Tools and repository

Teams are expected to use agentic coding tools to research methods, draft and revise code, create tests, and diagnose failures. The team remains responsible for every assumption, the algorithm logic, feasibility, and the interpretation of results. Tool choice and prompt volume are not graded, transcripts are not required, and the README names the tools used. Documented commands for solving, checking, and comparing are sufficient, and a Marimo or other visual interface is optional. Grading must not require a paid AI service, a private account, or any undisclosed external system.

The repository contains all implementation code, the README, and a Pixi manifest and lock file. It also contains the tests, one documented command that runs the tests, and one documented command that regenerates every submitted comparison and plot. External code and data require attribution. Secrets, personal data, caches, and machine-specific absolute paths are excluded, and submitted results match the submitted repository version.

3 Milestones and deliverables

The project has three milestones. Table 2 lists what each one must contain, and the course schedule and Blackboard give the dates. For the deterministic prototype and the final submission, teams submit the repository link, the commit hash of the version shown, and the video link through the matching Blackboard assignment. A repository may be public, or private if `freem028@ua.edu` has access to it through grading. Videos are unlisted YouTube videos that are viewable without requesting access, and the instructor stops viewing a recording at its time limit.

Table 2: Deliverables by milestone.

Milestone	Required contents
Team email	An email to `freem028@ua.edu` that lists the team members and the chosen option. It carries no points.

Milestone	Required contents
Deterministic prototype	The repository and a video of at most 10 minutes. The video runs the documented command on a fresh instance (a supplied instance directory copied to a new path), runs both methods, shows the checker confirming both decisions, and gives an initial comparison. It explains the decision in the team's own words and names the uncertain input the team plans to vary. Each member walks through one component that a coding agent drafted, i.e., what it does, one thing that was wrong or unclear in the agent's version, and how the team checked it. The instructor shares prototype recordings with the class.
Final submission	The repository and a video of at most 15 minutes that stays available through grading. The video shows a fresh instance, both methods and their decisions, the checker, the deterministic comparison, the stress-test results, the recommendation, and a limitation that could change it. Each member again walks through one agent-drafted component as in the prototype, choosing a different component. If a method changed after the prototype, the README explains why.
Peer evaluations	A confidential evaluation after the prototype and after the final submission (Section 4.4).

4 Grading

The project is worth 70 course points. The deterministic prototype is worth 20 team points, the final submission is worth 40 team points, and individual contribution is worth 10 points. Team points are shared by every member. Peer ratings affect only the individual contribution points, and they never multiply the team score. When a `decision.json` that a submitted method wrote fails the instructor's checker, on a supplied case or in the held-out run, the method criterion and the comparison criterion each earn at most half their points in that submission, because a comparison of invalid decisions does not establish operating performance.

4.1 Deterministic prototype rubric

Criterion	Points	Full-credit evidence
Ingestion and application behavior	3	The tool reads a fresh supplied instance without code edits, applies the four input checks, reports failures clearly, and exports results.
Two method implementations	6	The rule to beat and the researched method run, enforce the required constraints, report their stopping rule and status accurately, and can be explained from their steps through the code.
Independent validation and tests	4	A separate checker verifies feasibility and recomputes measures, and the three required tests pass.
Initial comparison	3	The comparison uses at least three common instances, the primary and secondary measures, and runtime, and it avoids claims the evidence does not support.
Recording	4	The recording stays within 10 minutes, shows fresh input and actual outputs, and includes every member's walk-through of an agent-drafted component, naming what was wrong and how it was checked.
Total	**20**	

4.2 Final submission rubric

Criterion	Points	Full-credit evidence
Application and reproducibility	6	The tool installs through Pixi, runs through documented commands, responds correctly to the held-out valid and malformed files, and exports results.
Method correctness	6	Both methods match their documented logic and enforce the required constraints. Stopping rules, seeds, solver status, and limitations are reported accurately.
Held-out performance against the rule to beat	2	On the held-out case, the researched method, run by the documented command with default settings within its time limit, writes a `decision.json` that passes the instructor's checker and has a primary measure better than the rule to beat's by more than 0.001. Otherwise this criterion earns 0 points.
Validation and tests	5	Independent feasibility and measure checks pass on supplied and held-out cases, and the three required tests pass.
Deterministic comparison	5	The comparison covers all four instances, reports the primary and secondary measures and runtime, examines how results change with instance size, and avoids universal claims.
Stress test	5	Thirty scenarios from a documented seed are saved before any method runs, both fixed plans are executed on the same scenarios by the brief's execution rule, and the mean, 90th percentile, constraint violations, and a per-scenario plot are reported for the primary and secondary measures.
Recommendation	3	The recommendation names the method, its expected benefit, the tradeoff accepted, and one condition under which it would change.
Final video	6	The video is accessible, stays within 15 minutes, demonstrates actual application behavior, connects evidence to the recommendation, and includes every member's walk-through of an agent-drafted component.
Code and documentation	2	All source and tests are submitted, and the README supports running and checking the tool and names the tools used.
Total	**40**	

4.3 What the instructor runs

The final-submission run

The README gives one command that takes an input directory and an output directory (e.g., `pixi run solve <input_dir> <output_dir>`), runs the researched method with its default settings, and writes `decision.json`. The instructor builds a clean environment from the Pixi files, runs the test command, runs the solve command on one supplied instance and compares the result with the submitted results, and then runs it on a held-out valid instance of size 60 (about 60 operations for option 2) and on one malformed instance.

- **Malformed input** (it fails one of the four checks in Section 2.2). The tool prints a message naming the file and column, writes no `decision.json`, and exits with a nonzero status.

- **Valid input for which the method finds nothing feasible.** The tool prints "no feasible decision found", writes no `decision.json`, and exits with status zero.
- **Any other valid input.** The tool writes `decision.json` and exits with status zero, including option 7's valid empty plan.

The whole command, including loading and writing, must finish within 10 minutes on the instructor's computer. The default settings fix the seed and stop on a count of iterations or evaluations, with the time limit only as a safety cap, so a rerun reproduces the submitted results.

The instructor's checker reads only `decision.json`. For options 1, 2, 4, and 6 it reads every start and completion, and each duration must match the input to within 10^{-8} time units, so reported times must not be rounded. For option 5 it reads each batch's orders, visits, and start, and for options 3 and 7 it reads the visits on each route. It recomputes every other time and every measure, ignores extra keys, and allows the same 10^{-8} tolerance at every window, shift, and availability end. Graded files keep the structure of the supplied files (unique identifiers, complete travel tables, and intervals that start before they end), and a malformed file breaks one of the four input checks.

4.4 Individual contribution and peer evaluation

After the prototype and after the final submission, every student submits a confidential evaluation of every team member, including themselves. The evaluation asks for specific evidence on five dimensions, which are contributing substantive work, working constructively with teammates, keeping commitments and the team on track, expecting quality in implementation, validation, and documentation, and developing and sharing relevant technical knowledge. Students also identify the components they personally implemented or reviewed.

Ratings use behavior descriptions from 1 to 5. A 1 describes repeated absence, unreliability, or unusable work, and a 2 describes inconsistent contribution that requires substantial reminders or rework. A 3 describes reliable completion of agreed work and the ability to explain it, a 4 describes strong contribution that also improves teammates' work, and a 5 describes sustained exceptional contribution while helping the whole team succeed.

Self-ratings support reflection and discrepancy checks but do not enter the grade calculation. Let p_{it} be student i's mean rating from teammates across the five dimensions at checkpoint t, and let $\bar{p}_t$ be the mean of those peer averages within the team. The provisional contribution factor is

$$f_{it} = \min\left\{1, \frac{p_{it}}{\bar{p}_t}\right\},$$

and the individual contribution score, out of 10 points, is

$$C_i = 4f_{i,\text{Prototype}} + 6f_{i,\text{Final}}.$$

For example, in a team whose peer means are 4.2, 3.9, and 2.7, the team mean is 3.6, the first two students earn all 4 prototype points, and the third earns $4(2.7/3.6) = 3.00$. The cap at 1 means a student at or above the team mean receives full credit, and a student below the mean can recover through stronger contribution before the final checkpoint.

A student who works alone submits no peer evaluation and receives all 10 contribution points, subject to the same review of repository and recording evidence. In a two-student team, each student's peer mean comes from one teammate. The calculation produces a provisional score, not an automatic judgment, and the instructor reviews every factor below 1 against the repository and recording evidence, allowing the affected student to respond, before it affects a grade.

5 Data packages

Each option has one data ZIP, and every ZIP has the layout in Table 5. The ZIP's DATA_DICTIONARY.md defines every column, the units, and the conventions for that option, and each instance directory contains the CSV inputs, a `schema.json` file listing every column's type and whether it may be empty, and an `instance.json` file recording the time unit and the condition the instance tests. The option brief is supplied beside this guide and again inside the option's data ZIP.

Table 5: Contents of each data ZIP.

Directory	Size	Intended use
`hand_checkable/`	6	Trace the rule to beat by hand and build the first test.
`worked-example/`	6	The rule to beat applied to the hand-checkable case, with every measure except computation time recomputed. Its `solution.json` shows the structure of `decision.json`.
`development_small/`	15	Debug methods and the checker.
`development_medium/`	45	Compare methods at a moderate size and run the stress test.
`development_large/`	120	Test scale and runtime.
`edge_case/`	10	Test the documented infeasible, malformed, or empty condition.
Held-out (not supplied)	60	Used by the instructor at the final submission.

Size counts jobs, customers, activities, orders, or visits, except in option 2, where it is an approximate number of operations.